

0CTF/TCTF 2020 Crypto - babyring

deuterium — 2020-06-30

Canonical HTML: <https://deut-erium.github.io/WriteUps/2020/tctf/crypto/babyring/2020-06-30-TCTF-2020-Quals-babyring.html>

Abstract

CTF writeup: 0CTF/TCTF 2020's babyring makes RC4 run in circles. Reduce the ring check to XOR equations and let SageMath choose RSA inputs that close the loop.

Babyring

Description

```
nc pwnable.org 10001
```

Files

- [ring.tar.gz](#)
 - [release/task.py](#)

task.py reads (trimming most part)

```
#!/usr/bin/python2
import os, random, sys, string
from hashlib import sha256
from struct import pack, unpack
import SocketServer
from Crypto.Cipher import ARC4

from flag import flag

K = 64

def gen():
    from Crypto.Util.number import getStrongPrime
    e = 65537
    Ns = []
    for i in range(K):
        p = getStrongPrime(2048)
        q = getStrongPrime(2048)
        Ns.append(p*q)
    return e, Ns

e, Ns = 65537, #list trimmed[...]
class Task(SocketServer.BaseRequestHandler):
    def proof_of_work(self):
        proof = ''.join([random.choice(string.ascii_letters+string.digits) for _ in
xrange(20)])
        digest = sha256(proof).hexdigest()
```

```

        self.request.send("sha256(XXXX+%s) == %s\n" % (proof[4:],digest))
        self.request.send('Give me XXXX:')
        x = self.request.recv(10)
        x = x.strip()
        if len(x) != 4 or sha256(x+proof[4:]).hexdigest() != digest:
            return False
        return True

def handle(self):
    if not self.proof_of_work():
        return
    self.request.settimeout(3)
    try:
        self.request.sendall("message: ")
        msg = self.request.recv(0x40).strip()
        ys = []
        for i in range(K):
            self.request.sendall("x%d: " % i)
            x = int(self.request.recv(0x40).strip())
            ys.append(pow(x,e,Ns[i]))
        self.request.sendall("v: ")
        v = int(self.request.recv(0x40).strip())

        key = sha256(msg).digest()[:16]
        E = ARC4.new(key)
        cur = v
        for i in range(K):
            pt = (ys[i]^cur)%(1<<64)
            ct = unpack('Q', E.encrypt(pack('Q',pt)))[0]
            cur = ct

        if cur == v:
            self.request.sendall("%s\n" % flag)
            self.request.sendall("fin\n")
    finally:
        self.request.close()

class ThreadedServer(SocketServer.ThreadingMixIn, SocketServer.TCPServer):
    pass

if __name__ == "__main__":
    HOST, PORT = '0.0.0.0', 10001
    server = ThreadedServer((HOST, PORT), Task)
    server.allow_reuse_address = True
    server.serve_forever()

```

The first part is obviously proof of work, in which we have to find 4 bytes XXXX such that $\text{sha256}(\text{XXXX} + 16\text{-char-val}) = \text{sha256_hash}$ for provided 16-char-val postfix and sha256_hash. Which is easy to solve

using permutations may not always work (in case of repeated characters), earlier I used combinations with replacement which had some weird issues which I could not debug. “python from hashlib import sha256 import string from itertools import permutations as take CHARSET_SHA = string.printable[:62].encode() #0-9a-zA-Z as in challenge

```
def pow`sha(postfix, hash`val): for prefix in take(CHARSET`SHA, 4): prefix`bytes =
bytes(prefix) shaa = sha256(prefix`bytes+postfix).hexdigest() if shaa == SHA`256`HASH:
return prefix`bytes
```

```
HOST, PORT = "pwnable.org", 10001 REM = remote(HOST, PORT) SHA`CHALL =
REM.recvuntil(b'XXXX:') #print(SHA`CHALL.decode()) SHA`256`HASH =
re.search(b"[0-9a-f]-64'",SHA`CHALL).group(0).decode() POSTFIX`STR = re.search(b"[0-
9a-zA-Z]-16'",SHA`CHALL).group(0) PREFIX`CHALL = pow`sha(POSTFIX`STR,
SHA`256`HASH) REM.send(PREFIX`CHALL)
```

Now comes the main part of the challenge, in which we have to provide 64 `xi` values a message `msg` and a value `v`
The `msg` is sha256 hashed and first 16 bytes are taken to form the `key` for `ARC4`
The value `v` is XORed with last 64 bits of `pow(x[i], e, Ns[i])` and then `ARC4` encrypted. The goal is to produce the final value equal to the input `v`.

Since `ARC4` is simply a stream cipher, and encryption is just XORing the plaintext with a keystream, our final value `cur` is essentially
 $v \oplus y[0] \oplus \dots \oplus y[63] \oplus xors[0] \oplus xors[1] \dots \oplus xors[63]$, where `ys[0..63]` are the last 64 bits of the respective `y[0..63]` and `xors[0]xors[1]...xors[63]` part is essentially dependent on `key` and an invariant for a given `key`, lets call it `invariant(key)` (bye bye `ARC4`).
All we need to do is to find `x[0..63]` such that $y[0] \oplus y[1] \dots \oplus y[63] == invariant(key)$ and we will have `cur==v` for all `v` as a consequence.

```
```python
from Crypto.Cipher import ARC4
from hashlib import sha256
from struct import pack, unpack

def encrypt_64(v,key,y):
 E = ARC4.new(key)
 cur = v
 for i in range(64):
 pt = (cur^y[i])%(1<<64)
 ct = unpack('Q',E.encrypt(pack('Q',pt)))[0]
 cur = ct
 return cur

def invariant(key):
 key_val = sha256(key).digest()[:16]
 return encrypt_64(0,key_val,[0 for i in range(64)])

print(invariant(b'aaa'))
911494890333775973
```

One could simply put  $x[i]$  as some value such that  $y[i] == invariant$  and all other  $x_s == 0$  but only if one could solve ANY of the RSA by factoring 4096 bit  $N_s$ , which is obviously not feasible!

Not knowing much linear algebra, I found this [stackexchange post](#) and [this](#) showing all that needs to be done is to have a set of 64 64-bit vectors, and we can represent any 64 bit value using xor of a subset of the vectors. Taking the corresponding  $y$ s for  $x_s = 2$  for all  $i$ , and solving the subset for given invariant, we will only set  $x[i] = 2$  in the  $i$  in subset else  $x[i]=0$  (to have no effect).

Sagemath ftw! [CoCalc](#) for the poor

```
#Ns not shown here
last_64 = [pow(2,e,i)%(1<<64) for i in Ns]
invariant = 911494890333775973 #for msg = b'aaa'
I = GF(2**64)
last_64_mat = [list(map(int, bin(i)[2:].zfill(64))) for i in last_64]
mat = matrix(I,last_64_mat)
invariant_vec = list(map(int, bin(invariant)[2:].zfill(64)))
invariant_vec = matrix(I,invariant_vec)
op = mat.solve_left(invariant_vec)
print(op[0])
```

Awesome! we have (0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1) as our output vector, we just have to return  $x_i$  as  $2*op$  and we are done ;)

```
xs = (0, 1, 0, 1, 0, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1)
xs = [2*i for i in xs]
REM.recv()
REM.send(b'aaa') #message
for i in range(64):
 REM.send(str(xs[i]).encode()) #xi's
 REM.recv()

REM.send(b'0') #v any v would do the job ;)
REM.recv()
#flag{babbbcbdbefbgbhbibjbkblmbnbnobpbqbrbsbtbubvbwxbxby}
```

Unorganized code in files [solve.py](#), [part2.sage](#) and [test.py](#)

**flag-babbbcbdbefbgbhbibjbkblmbnbnobpbqbrbsbtbubvbwxbxby"**

| Interactive controls are omitted from this PDF. See the canonical HTML version.